

Course 2: Supervised Learning



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

2026-08-24

Course 2: Supervised Learning

Course 2: Supervised Learning



Summary

Last session

- AI definition
- Applications & Open Issues
- Deep learning
- Foundation models

Today's session

- Learning from labeled examples
- Challenges of supervised learning
- Evaluation: ground truth and metrics

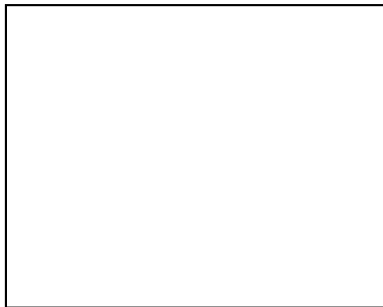
Last session

- 1 AI definition
- 2 Applications & Open Issues
- 3 Deep learning
- 4 Foundation models

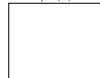
Today's session

- Learning from labeled examples
- Challenges of supervised learning
- Evaluation: ground truth and metrics

Vector space ($\mathbb{R}^d$)



Notations

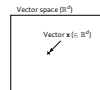


Notations used throughout the course. $\mathbb{R}^d$ is the vector space of real values in dimension d ; a single example is a vector $\mathbf{x} \in \mathbb{R}^d$, i.e. a list of d numbers, and X denotes the set of all the examples we have.

The mental image to keep is the one on the slide: **one example = one point in a space**. The rectangle is a 2D drawing of that space, but d is usually much larger (hundreds or thousands), which is exactly what will cause trouble later (see the curse of dimensionality).

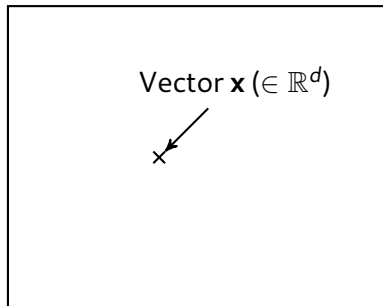
Concretely, d is the number of **features** describing one example: for a table, one column per feature; for a 32×32 colour image seen as raw pixels, $d = 32 \times 32 \times 3 = 3072$. Two examples that are similar in the real world should ideally be close in this space; whether they actually are depends entirely on how the features were built, which is the subject of the next slides.

Vocabulary: n (number of examples, the size of X) and d (dimension, the number of features) are the two quantities you should always know about a dataset before doing anything with it.



Notations

Vector space ($\mathbb{R}^d$)

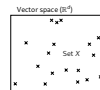


Notations used throughout the course. $\mathbb{R}^d$ is the vector space of real values in dimension d ; a single example is a vector $\mathbf{x} \in \mathbb{R}^d$, i.e. a list of d numbers, and X denotes the set of all the examples we have.

The mental image to keep is the one on the slide: **one example = one point in a space**. The rectangle is a 2D drawing of that space, but d is usually much larger (hundreds or thousands), which is exactly what will cause trouble later (see the curse of dimensionality).

Concretely, d is the number of **features** describing one example: for a table, one column per feature; for a 32×32 colour image seen as raw pixels, $d = 32 \times 32 \times 3 = 3072$. Two examples that are similar in the real world should ideally be close in this space; whether they actually are depends entirely on how the features were built, which is the subject of the next slides.

Vocabulary: n (number of examples, the size of X) and d (dimension, the number of features) are the two quantities you should always know about a dataset before doing anything with it.



Notations

Vector space ($\mathbb{R}^d$)



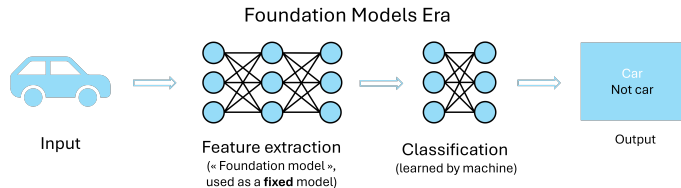
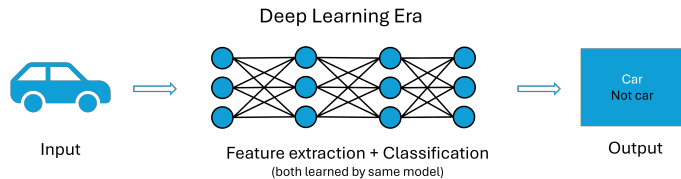
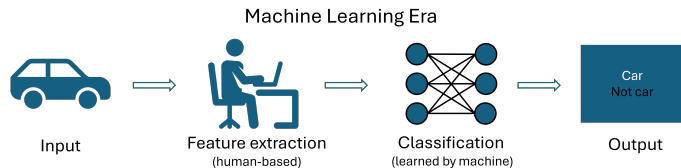
Notations used throughout the course. $\mathbb{R}^d$ is the vector space of real values in dimension d ; a single example is a vector $\mathbf{x} \in \mathbb{R}^d$, i.e. a list of d numbers, and X denotes the set of all the examples we have.

The mental image to keep is the one on the slide: **one example = one point in a space**. The rectangle is a 2D drawing of that space, but d is usually much larger (hundreds or thousands), which is exactly what will cause trouble later (see the curse of dimensionality).

Concretely, d is the number of **features** describing one example: for a table, one column per feature; for a 32×32 colour image seen as raw pixels, $d = 32 \times 32 \times 3 = 3072$. Two examples that are similar in the real world should ideally be close in this space; whether they actually are depends entirely on how the features were built, which is the subject of the next slides.

Vocabulary: n (number of examples, the size of X) and d (dimension, the number of features) are the two quantities you should always know about a dataset before doing anything with it.

What is the vector x ? (1/2)

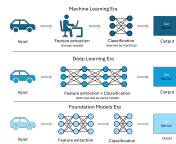


2026-08-24

Course 2: Supervised Learning

What is the vector x ? (1/2)

What is the vector x ? (1/2)



What is the vector x ? (2/2)

Traditional Machine Learning

→ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

Deep Learning

→ x is the data, or a small transformation of the data

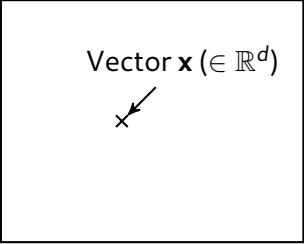
The era of Foundation models

→ x is the projection of data in an **embedding space**

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable

Vector space ($\mathbb{R}^d$)

Vector $\mathbf{x} (\in \mathbb{R}^d)$



2026-08-24

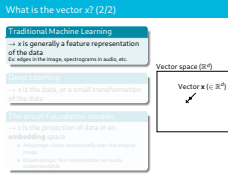
Course 2: Supervised Learning

What is the vector x ? (2/2)

In traditional Machine Learning, data is generally a feature representation of the data (i.e. the edges or the colorimetry levels in an image, or spectrograms in audio).

A first shift appears with Deep Learning, where data is fed as it is to models, allowing to alleviate the complex, time-consuming and hand-crafted feature engineering, and leading to impressive performance in image processing, the drawback being a need for large amounts of data and computing power.

Finally, a recent shift appear with larger and more sophisticated models ("Foundation Models"), where they can be used as a first pre-processing step. The resulting representations are called "embeddings", and are vectors in the latent space of the model, i.e. the input vectors (image, text, ...) that have been mapped in a lower dimensional space by a function $f : R^d \rightarrow R^l$. Embeddings are usually richer semantically and easier to manipulate for a downstream task (e.g. classification).



What is the vector x ? (2/2)

Traditional Machine Learning

→ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

Deep Learning

→ x is the data, or a small transformation of the data

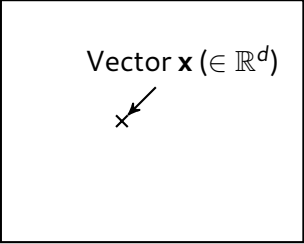
The era of Foundation models

→ x is the projection of data in an **embedding space**

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable

Vector space ($\mathbb{R}^d$)

Vector $\mathbf{x} (\in \mathbb{R}^d)$



2026-08-24

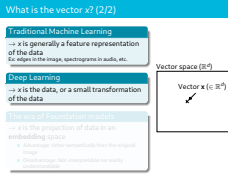
Course 2: Supervised Learning

What is the vector x ? (2/2)

In traditional Machine Learning, data is generally a feature representation of the data (i.e. the edges or the colorimetry levels in an image, or spectrograms in audio).

A first shift appears with Deep Learning, where data is fed as it is to models, allowing to alleviate the complex, time-consuming and hand-crafted feature engineering, and leading to impressive performance in image processing, the drawback being a need for large amounts of data and computing power.

Finally, a recent shift appear with larger and more sophisticated models ("Foundation Models"), where they can be used as a first pre-processing step. The resulting representations are called "embeddings", and are vectors in the latent space of the model, i.e. the input vectors (image, text, ...) that have been mapped in a lower dimensional space by a function $f : R^d \rightarrow R^l$. Embeddings are usually richer semantically and easier to manipulate for a downstream task (e.g. classification).



What is the vector x ? (2/2)

Traditional Machine Learning

→ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

Deep Learning

→ x is the data, or a small transformation of the data

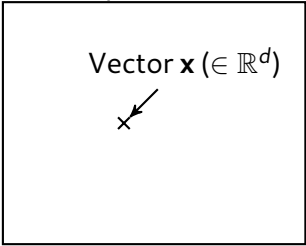
The era of Foundation models

→ x is the projection of data in an **embedding space**

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable

Vector space ($\mathbb{R}^d$)

Vector $\mathbf{x} (\in \mathbb{R}^d)$



2026-08-24

Course 2: Supervised Learning

What is the vector x ? (2/2)

What is the vector x ? (2/2)

Traditional Machine Learning → x is generally a feature representation of the data Ex: edges in the image, spectrograms in audio, etc.	Vector space ($\mathbb{R}^d$) Vector $\mathbf{x} (\in \mathbb{R}^d)$
Deep Learning → x is the data, or a small transformation of the data	
The era of Foundation models → x is the projection of data in an embedding space ■ Advantage: richer semantically than the original image ■ Disadvantage: Not interpretable nor easily understandable	

In traditional Machine Learning, data is generally a feature representation of the data (i.e. the edges or the colorimetry levels in an image, or spectrograms in audio).

A first shift appears with Deep Learning, where data is fed as it is to models, allowing to alleviate the complex, time-consuming and hand-crafted feature engineering, and leading to impressive performance in image processing, the drawback being a need for large amounts of data and computing power.

Finally, a recent shift appear with larger and more sophisticated models ("Foundation Models"), where they can be used as a first pre-processing step. The resulting representations are called "embeddings", and are vectors in the latent space of the model, i.e. the input vectors (image, text, ...) that have been mapped in a lower dimensional space by a function $f : R^d \rightarrow R^l$. Embeddings are usually richer semantically and easier to manipulate for a downstream task (e.g. classification).

1 Learning from labeled examples

2 Challenges of supervised learning

3 Evaluation: ground truth and metrics

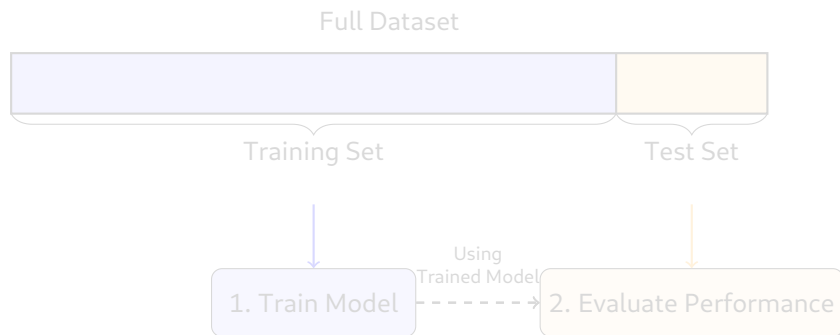
Learning: Training and Test sets

The Training Set

Data used by the model to **learn** from examples.

The Test Set

A smaller, hold-out portion that the model has **never seen**. Used to **evaluate** its performance.

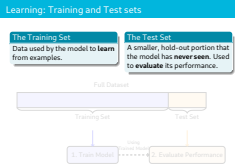


2026-08-24

Course 2: Supervised Learning

└ Learning from labeled examples

└ Learning: Training and Test sets



Rule number one: **never evaluate a model on the data it was trained on**. Reproducing answers already seen is memorisation, and a lookup table does it perfectly. What we want is **generalisation**: being right on examples unseen during training, since those are the ones the model will meet once deployed. The test set simulates that situation.

In practice: 80/20 or 70/30, and the split must be **random** (ex: if your file is sorted by class, the last 20% may contain a single class). In `sklearn`, `train_test_split` does it, and fixing `random_state` makes the split reproducible, which is needed to compare models fairly.

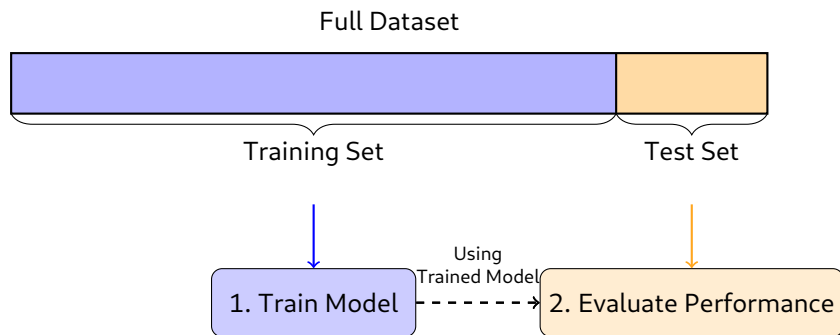
The cardinal sin is **data leakage**: any information from the test set reaching the training procedure (e.g., duplicated examples).

The Training Set

Data used by the model to **learn** from examples.

The Test Set

A smaller, hold-out portion that the model has **never seen**. Used to **evaluate** its performance.

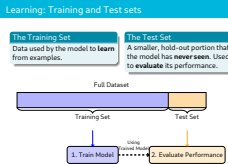


2026-08-24

Course 2: Supervised Learning

Learning from labeled examples

Learning: Training and Test sets



Rule number one: **never evaluate a model on the data it was trained on**. Reproducing answers already seen is memorisation, and a lookup table does it perfectly. What we want is **generalisation**: being right on examples unseen during training, since those are the ones the model will meet once deployed. The test set simulates that situation.

In practice: 80/20 or 70/30, and the split must be **random** (ex: if your file is sorted by class, the last 20% may contain a single class). In `sklearn`, `train_test_split` does it, and fixing `random_state` makes the split reproducible, which is needed to compare models fairly.

The cardinal sin is **data leakage**: any information from the test set reaching the training procedure (e.g., duplicated examples).

Definition

Given a training set, composed of:

- **x**: inputs (raw signals or feature vectors (e.g. embeddings))
- **y**: **labels** (annotated by humans)

Learn:

- a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
 $\Rightarrow f()$ is **learned** by the Machine Learning algorithm
- Ideally, $f()$ should **generalize** ($\neq$ memorize) to unlabeled examples.

2026-08-24

Course 2: Supervised Learning

└ Learning from labeled examples

└ Supervised learning

Supervised learning

Definition
Given a training set, composed of:
■ **x**: inputs (raw signals or feature vectors (e.g. embeddings))
■ **y**: **labels** (annotated by humans)
Learn:
■ a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
 $\Rightarrow f()$ is learned by the Machine Learning algorithm
■ Ideally, $f()$ should generalize ($\neq$ memorize) to unlabeled examples

In supervised learning, we are given **pairs** $(\mathbf{x}, \mathbf{y})$: an input and the answer expected for it. In this context, someone (typically a human expert) has annotated the data with the correct answers.

The goal is to find a function f such that $f(\mathbf{x}) \approx \mathbf{y}$ on the training pairs. Note the " $\approx$ ": we are **not** looking for a perfect equality in most cases, because that generally lead to overfitting (see later): a function that fit so much the training answers that it also fit irrelevant components, leading in turn to poor generalization on examples that were never annotated.

A remark on algorithms: f is chosen inside a restricted **family** of functions (linear functions, decision trees, neural networks...), fixed by the algorithm you pick; "learning" means selecting the best member of that family, by adjusting its parameters.

Producing the labels $\mathbf{y}$ is usually the bottleneck of a real project: it is slow, expensive, requires expertise, and its quality directly caps the quality of the model.

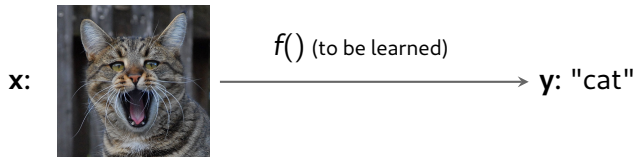
Definition

Given a training set, composed of:

- **x**: inputs (raw signals or feature vectors (e.g. embeddings))
- **y**: **labels** (annotated by humans)

Learn:

- a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
 $\Rightarrow f()$ is **learned** by the Machine Learning algorithm
- Ideally, $f()$ should **generalize** ($\neq$ memorize) to unlabeled examples.



2026-08-24

Course 2: Supervised Learning

└ Learning from labeled examples

└ Supervised learning

Supervised learning

Definition

Given a training set, composed of:

- **x**: inputs (raw signals or feature vectors (e.g. embeddings))
- **y**: **labels** (annotated by humans)

Learn:

- a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
 $\Rightarrow f()$ is **learned** by the Machine Learning algorithm
- Ideally, $f()$ should **generalize** ($\neq$ memorize) to unlabeled examples.

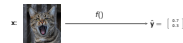
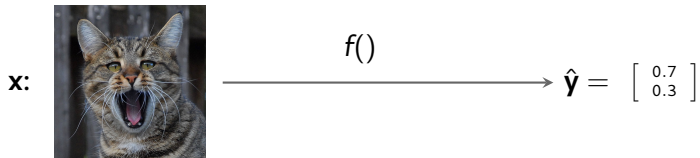
x:  $\xrightarrow{f() \text{ (to be learned)}}$ **y: "cat"**

In supervised learning, we are given **pairs** $(\mathbf{x}, \mathbf{y})$: an input and the answer expected for it. In this context, someone (typically a human expert) has annotated the data with the correct answers.

The goal is to find a function f such that $f(\mathbf{x}) \approx \mathbf{y}$ on the training pairs. Note the " $\approx$ ": we are **not** looking for a perfect equality in most cases, because that generally lead to overfitting (see later): a function that fit so much the training answers that it also fit irrelevant components, leading in turn to poor generalization on examples that were never annotated.

A remark on algorithms: f is chosen inside a restricted **family** of functions (linear functions, decision trees, neural networks...), fixed by the algorithm you pick; "learning" means selecting the best member of that family, by adjusting its parameters.

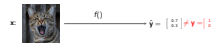
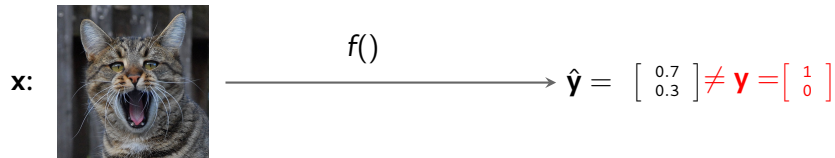
Producing the labels $\mathbf{y}$ is usually the bottleneck of a real project: it is slow, expensive, requires expertise, and its quality directly caps the quality of the model.



Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification. With one-hot labels the latter reduces to $-\log$ of the score given to the correct class: small when the model is right, exploding when it is confidently wrong. It requires outputs behaving like probabilities, obtained with a **softmax**.

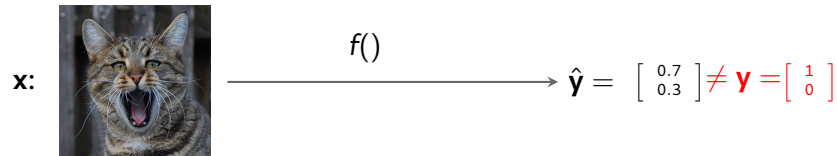
Training. Summed over all examples, the loss is a single number depending on the parameters of f ; learning is minimising it, typically by gradient descent (Session 1). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**



Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification. With one-hot labels the latter reduces to $-\log$ of the score given to the correct class: small when the model is right, exploding when it is confidently wrong. It requires outputs behaving like probabilities, obtained with a **softmax**.

Training. Summed over all examples, the loss is a single number depending on the parameters of f ; learning is minimising it, typically by gradient descent (Session 1). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**



Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
- Training consist in minimizing the loss!

2026-08-24

Course 2: Supervised Learning

Learning from labeled examples

Supervised learning: in practice



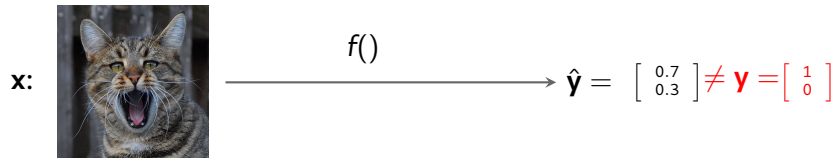
Loss

- Here, labels are encoded as one-hot vectors,
- We compute a loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
- Training consist in minimizing the loss!

Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

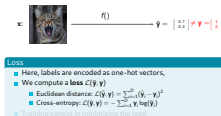
Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification. With one-hot labels the latter reduces to $-\log$ of the score given to the correct class: small when the model is right, exploding when it is confidently wrong. It requires outputs behaving like probabilities, obtained with a **softmax**.

Training. Summed over all examples, the loss is a single number depending on the parameters of f ; learning is minimising it, typically by gradient descent (Session 1). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**



Loss

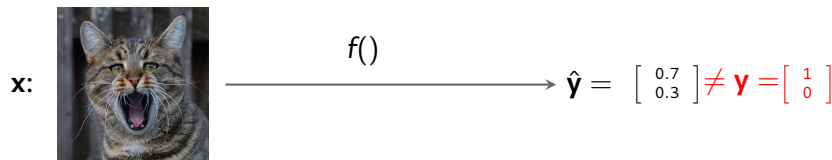
- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
 - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^D (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
 - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^D \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!



Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification. With one-hot labels the latter reduces to $-\log$ of the score given to the correct class: small when the model is right, exploding when it is confidently wrong. It requires outputs behaving like probabilities, obtained with a **softmax**.

Training. Summed over all examples, the loss is a single number depending on the parameters of f ; learning is minimising it, typically by gradient descent (Session 1). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**



Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
 - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^D (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
 - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^D \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!
⇒ Here, one can use gradient descent (see class 1.)

2026-08-24

Course 2: Supervised Learning

Learning from labeled examples

Supervised learning: in practice

Supervised learning: in practice

$\mathbf{x}$:  $\xrightarrow{f()}$ $\hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

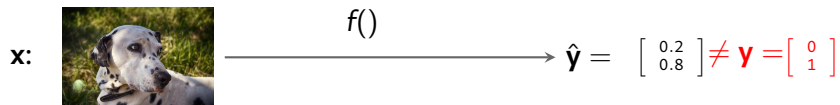
Loss

- Here, labels are encoded as one-hot vectors,
- We compute a loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
 - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^D (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
 - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^D \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss
⇒ Here, one can use gradient descent (see class 1.)

Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification. With one-hot labels the latter reduces to $-\log$ of the score given to the correct class: small when the model is right, exploding when it is confidently wrong. It requires outputs behaving like probabilities, obtained with a **softmax**.

Training. Summed over all examples, the loss is a single number depending on the parameters of f ; learning is minimising it, typically by gradient descent (Session 1). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**



Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
 - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^D (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
 - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^D \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!
 $\Rightarrow$ Here, one can use gradient descent (see class 1.)

2026-08-24

Course 2: Supervised Learning

Learning from labeled examples

Supervised learning: in practice

Supervised learning: in practice

$\mathbf{x}$:  $\xrightarrow{f()}$ $\hat{\mathbf{y}} = \begin{bmatrix} 0.2 \\ 0.8 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
 - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^D (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
 - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^D \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!
 $\Rightarrow$ Here, one can use gradient descent (see class 1.)

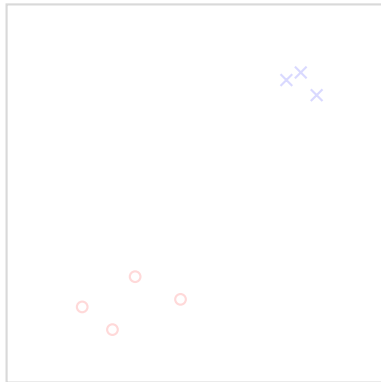
Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification. With one-hot labels the latter reduces to $-\log$ of the score given to the correct class: small when the model is right, exploding when it is confidently wrong. It requires outputs behaving like probabilities, obtained with a **softmax**.

Training. Summed over all examples, the loss is a single number depending on the parameters of f ; learning is minimising it, typically by gradient descent (Session 1). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**

Examples

- **Classification** (y is categorical)
- **Regression** (y is scalar)
- Tons of applications:
 - Pattern recognition,
 - Prediction...



2026-08-24

Course 2: Supervised Learning

└ Learning from labeled examples

└ Supervised learning

Supervised learning

Examples

- Classification (y is categorical)
- Regression (y is scalar)
- Tons of applications:
 - Pattern recognition,
 - Prediction...



Two families, depending on the **nature of y** . **Classification:** y is categorical, taking values in a finite unordered set (apple vs. orange, spam vs. legitimate, one of ten digits). **Regression:** y is a continuous scalar (a price, tomorrow's temperature).

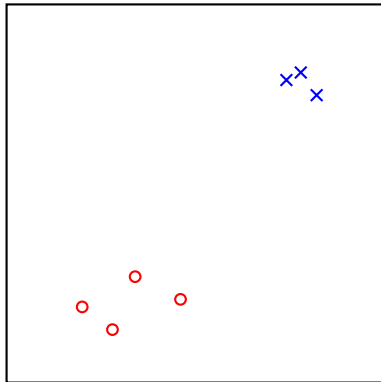
Identify which one your project is, as it determines the algorithms, the losses and the metrics. The metrics at the end of this course (accuracy, precision, recall) are specific to classification; regression uses the mean squared or mean absolute error.

On the figure, red circles and blue crosses are the training examples, drawn as points in the 2D space of the first slide. Learning a classifier amounts to **cutting the plane into regions**, one predicted red and the other blue; a new point is classified by the region it falls into.

Once again, learning is not memorising: the question is not whether the boundary separates *these* points (many do) but whether it is still right on points we have not seen.

Examples

- **Classification** (y is categorical)
- **Regression** (y is scalar)
- Tons of applications:
 - Pattern recognition,
 - Prediction...

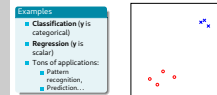


2026-08-24

Course 2: Supervised Learning

└ Learning from labeled examples

└ Supervised learning



Two families, depending on the **nature of y** . **Classification:** y is categorical, taking values in a finite unordered set (apple vs. orange, spam vs. legitimate, one of ten digits). **Regression:** y is a continuous scalar (a price, tomorrow's temperature).

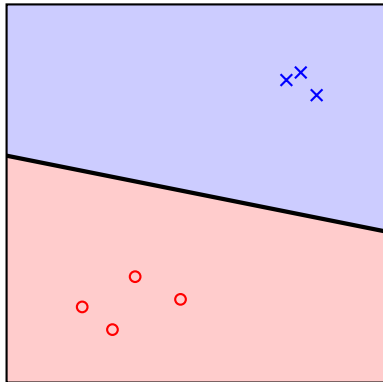
Identify which one your project is, as it determines the algorithms, the losses and the metrics. The metrics at the end of this course (accuracy, precision, recall) are specific to classification; regression uses the mean squared or mean absolute error.

On the figure, red circles and blue crosses are the training examples, drawn as points in the 2D space of the first slide. Learning a classifier amounts to **cutting the plane into regions**, one predicted red and the other blue; a new point is classified by the region it falls into.

Once again, learning is not memorising: the question is not whether the boundary separates *these* points (many do) but whether it is still right on points we have not seen.

Examples

- **Classification** (y is categorical)
- **Regression** (y is scalar)
- Tons of applications:
 - Pattern recognition,
 - Prediction...



2026-08-24

Course 2: Supervised Learning

└ Learning from labeled examples

└ Supervised learning



Two families, depending on the **nature of y** . **Classification:** y is categorical, taking values in a finite unordered set (apple vs. orange, spam vs. legitimate, one of ten digits). **Regression:** y is a continuous scalar (a price, tomorrow's temperature).

Identify which one your project is, as it determines the algorithms, the losses and the metrics. The metrics at the end of this course (accuracy, precision, recall) are specific to classification; regression uses the mean squared or mean absolute error.

On the figure, red circles and blue crosses are the training examples, drawn as points in the 2D space of the first slide. Learning a classifier amounts to **cutting the plane into regions**, one predicted red and the other blue; a new point is classified by the region it falls into.

Once again, learning is not memorising: the question is not whether the boundary separates *these* points (many do) but whether it is still right on points we have not seen.

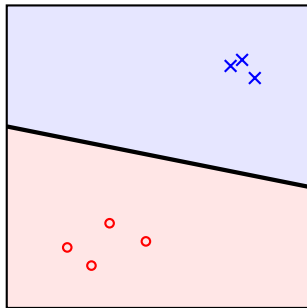
1 Learning from labeled examples

2 Challenges of supervised learning

3 Evaluation: ground truth and metrics

An ill-defined problem

- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



2026-08-24

Course 2: Supervised Learning

└ Challenges of supervised learning

└ Infinite solutions and priors

Infinite solutions and priors

An ill-defined problem

- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



The problem is **ill-posed**. The three boundaries classify the training points equally well, and nothing in the data lets us prefer one. Yet they disagree on new points, as the “?” shows: depending on the boundary, that point is red or blue. A finite set of examples is consistent with an infinity of functions, and the best one (which would be right on all future inputs) is unreachable by construction (we don’t have the future examples!).

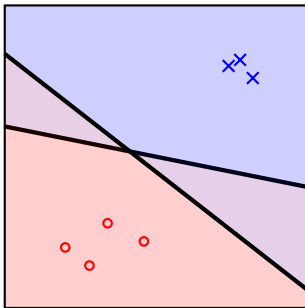
Two ways out:

1. **Add priors or constraints:** restrict the search to functions we consider reasonable. Choosing an algorithm does exactly this (a straight boundary, a smooth one, one far from the points (SVM), one made of axis-aligned thresholds (trees), etc). This is also why comparing algorithms is informative: each embodies a different prior.
2. **Add data:** every new labeled example eliminates the functions that contradict it.

Reflex for your project: if two very different models give the same test score, the problem is under-constrained, and the missing information should come from the data or from domain knowledge, not from a fancier algorithm.

An ill-defined problem

- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



2026-08-24

Course 2: Supervised Learning

└ Challenges of supervised learning

└ Infinite solutions and priors

Infinite solutions and priors

- An ill-defined problem
- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.

The problem is **ill-posed**. The three boundaries classify the training points equally well, and nothing in the data lets us prefer one. Yet they disagree on new points, as the “?” shows: depending on the boundary, that point is red or blue. A finite set of examples is consistent with an infinity of functions, and the best one (which would be right on all future inputs) is unreachable by construction (we don’t have the future examples!).

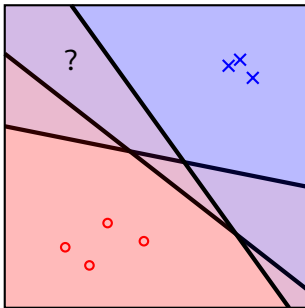
Two ways out:

1. **Add priors or constraints:** restrict the search to functions we consider reasonable. Choosing an algorithm does exactly this (a straight boundary, a smooth one, one far from the points (SVM), one made of axis-aligned thresholds (trees), etc). This is also why comparing algorithms is informative: each embodies a different prior.
2. **Add data:** every new labeled example eliminates the functions that contradict it.

Reflex for your project: if two very different models give the same test score, the problem is under-constrained, and the missing information should come from the data or from domain knowledge, not from a fancier algorithm.

An ill-defined problem

- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



2026-08-24

Course 2: Supervised Learning

└ Challenges of supervised learning

└ Infinite solutions and priors

Infinite solutions and priors

- An ill-defined problem
- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.

The problem is **ill-posed**. The three boundaries classify the training points equally well, and nothing in the data lets us prefer one. Yet they disagree on new points, as the “?” shows: depending on the boundary, that point is red or blue. A finite set of examples is consistent with an infinity of functions, and the best one (which would be right on all future inputs) is unreachable by construction (we don’t have the future examples!).

Two ways out:

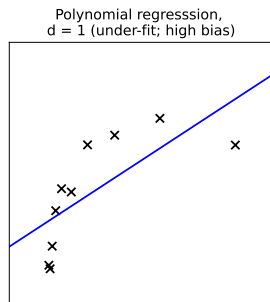
1. **Add priors or constraints:** restrict the search to functions we consider reasonable. Choosing an algorithm does exactly this (a straight boundary, a smooth one, one far from the points (SVM), one made of axis-aligned thresholds (trees), etc). This is also why comparing algorithms is informative: each embodies a different prior.
2. **Add data:** every new labeled example eliminates the functions that contradict it.

Reflex for your project: if two very different models give the same test score, the problem is under-constrained, and the missing information should come from the data or from domain knowledge, not from a fancier algorithm.

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.



- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.



The same points fitted by a polynomial of degree $d = 1, 2$ and 6 .

$d = 1$: a line cannot follow the data. Too rigid, wrong everywhere, and more points would not help. This is **high bias**: the error comes from an assumption built into the model itself.

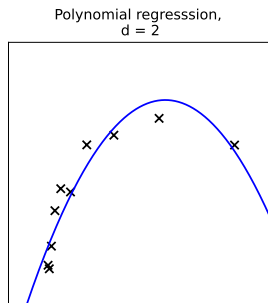
$d = 6$: the curve passes through every point... but is the worst of the three. Between the points it oscillates, so predictions elsewhere are unreliable, and moving a single training point would change it completely. This is **high variance**: the model absorbed the particularities, and the noise, of this training set. The trade-off: more capacity lowers the bias and raises the variance, so the total error is minimal at intermediate complexity. Hence the message of the slide. The same phenomenon appears everywhere with a different knob: degree of a polynomial, depth of a tree, number of neighbours k in a KNN (small k = high variance), size of a network.

(Rapid) Diagnostic: bad on *both* train and test = bias, model too simple; excellent on train and poor on test = variance, model too complex.

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.



- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.



The same points fitted by a polynomial of degree $d = 1, 2$ and 6 .

$d = 1$: a line cannot follow the data. Too rigid, wrong everywhere, and more points would not help. This is **high bias**: the error comes from an assumption built into the model itself.

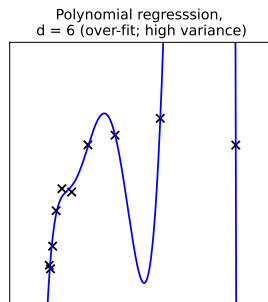
$d = 6$: the curve passes through every point... but is the worst of the three. Between the points it oscillates, so predictions elsewhere are unreliable, and moving a single training point would change it completely. This is **high variance**: the model absorbed the particularities, and the noise, of this training set. The trade-off: more capacity lowers the bias and raises the variance, so the total error is minimal at intermediate complexity. Hence the message of the slide. The same phenomenon appears everywhere with a different knob: degree of a polynomial, depth of a tree, number of neighbours k in a KNN (small k = high variance), size of a network.

(Rapid) Diagnostic: bad on *both* train and test = bias, model too simple; excellent on train and poor on test = variance, model too complex.

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.



- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.



The same points fitted by a polynomial of degree $d = 1, 2$ and 6 .

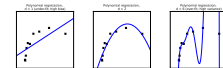
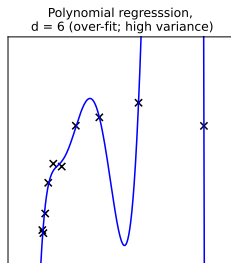
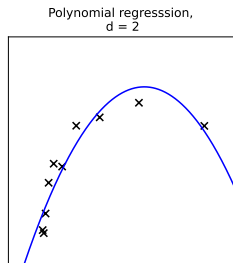
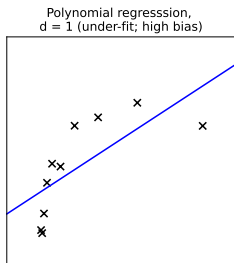
$d = 1$: a line cannot follow the data. Too rigid, wrong everywhere, and more points would not help. This is **high bias**: the error comes from an assumption built into the model itself.

$d = 6$: the curve passes through every point... but is the worst of the three. Between the points it oscillates, so predictions elsewhere are unreliable, and moving a single training point would change it completely. This is **high variance**: the model absorbed the particularities, and the noise, of this training set. The trade-off: more capacity lowers the bias and raises the variance, so the total error is minimal at intermediate complexity. Hence the message of the slide. The same phenomenon appears everywhere with a different knob: degree of a polynomial, depth of a tree, number of neighbours k in a KNN (small k = high variance), size of a network.

(Rapid) Diagnostic: bad on *both* train and test = bias, model too simple; excellent on train and poor on test = variance, model too complex.

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!



The same points fitted by a polynomial of degree $d = 1, 2$ and 6 .

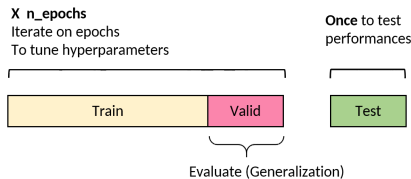
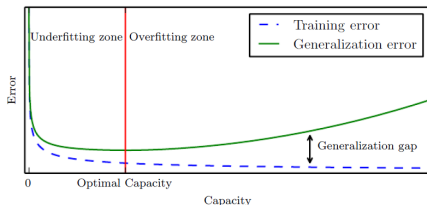
$d = 1$: a line cannot follow the data. Too rigid, wrong everywhere, and more points would not help. This is **high bias**: the error comes from an assumption built into the model itself.

$d = 6$: the curve passes through every point... but is the worst of the three. Between the points it oscillates, so predictions elsewhere are unreliable, and moving a single training point would change it completely. This is **high variance**: the model absorbed the particularities, and the noise, of this training set. The trade-off: more capacity lowers the bias and raises the variance, so the total error is minimal at intermediate complexity. Hence the message of the slide. The same phenomenon appears everywhere with a different knob: degree of a polynomial, depth of a tree, number of neighbours k in a KNN (small k = high variance), size of a network.

(Rapid) Diagnostic: bad on *both* train and test = bias, model too simple; excellent on train and poor on test = variance, model too complex.

Overfitting

Mimicking is not learning: **overfitting** problem.



To detect overfitting, you may use a validation set:

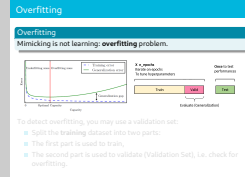
- Split the **training** dataset into two parts:
- The first part is used to train,
- The second part is used to validate (Validation Set), i.e. check for overfitting.

2026-08-24

Course 2: Supervised Learning

└ Challenges of supervised learning

└ Overfitting



The left figure is a **learning curve**: the error on the training set and on a held-out set, as a function of model complexity (read the axis as polynomial degree, tree depth, or training epochs).

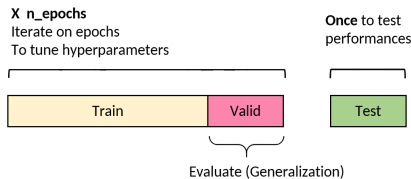
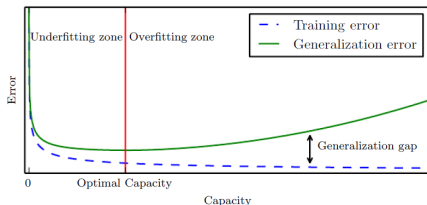
Both start high (**underfitting**: too simple to capture anything), then decrease, then **separate**: the training error keeps falling towards zero while the held-out error rises again. That divergence is **overfitting**, and the point you want is the minimum of the held-out curve. Practical rule: a growing train/test gap means overfitting (e.g. 100% on train and 65% on test is not learning, it is memorising).

Why a third subset? Detecting this needs unseen data, but if you use the test set to make choices (which degree, which k , when to stop) it is no longer unseen and its score becomes optimistic. So the training set is split again into a fit part and a **validation** set used for those choices; the test set is touched once, at the very end.

Typical: 60/20/20. When data is scarce, use **cross-validation** instead (`sklearn.model_selection.cross_val_score`).

Overfitting

Mimicking is not learning: **overfitting** problem.



To detect overfitting, you may use a validation set:

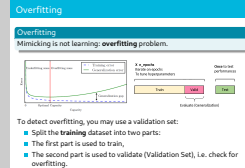
- Split the **training** dataset into two parts:
- The first part is used to train,
- The second part is used to validate (Validation Set), i.e. check for overfitting.

2026-08-24

Course 2: Supervised Learning

Challenges of supervised learning

Overfitting



The left figure is a **learning curve**: the error on the training set and on a held-out set, as a function of model complexity (read the axis as polynomial degree, tree depth, or training epochs).

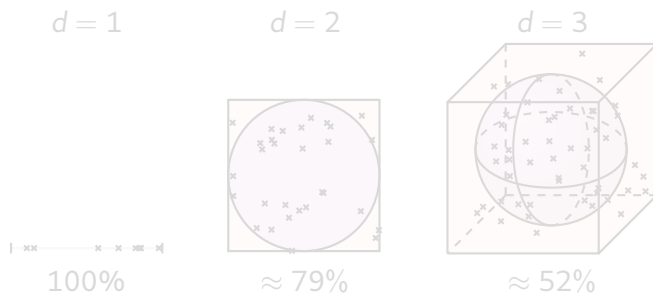
Both start high (**underfitting**: too simple to capture anything), then decrease, then **separate**: the training error keeps falling towards zero while the held-out error rises again. That divergence is **overfitting**, and the point you want is the minimum of the held-out curve. Practical rule: a growing train/test gap means overfitting (e.g. 100% on train and 65% on test is not learning, it is memorising).

Why a third subset? Detecting this needs unseen data, but if you use the test set to make choices (which degree, which k , when to stop) it is no longer unseen and its score becomes optimistic. So the training set is split again into a fit part and a **validation** set used for those choices; the test set is touched once, at the very end.

Typical: 60/20/20. When data is scarce, use **cross-validation** instead (`sklearn.model_selection.cross_val_score`).

Curse of dimensionality

- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.



$$V_d^S = \frac{\pi^{d/2} R^d}{\Gamma(d/2 + 1)} \text{ versus } V_d^C = (2R)^d$$

see <https://youtu.be/dZrGXYty3qc?t=533>

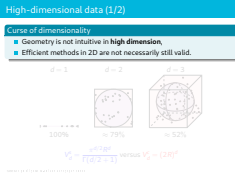
2026-08-24

Course 2: Supervised Learning

└ Challenges of supervised learning

└ High-dimensional data (1/2)

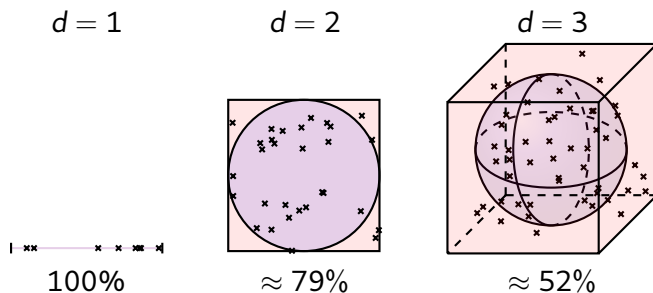
The point here is to show that when the dimension increases, the space tends to be more and more "empty". V_d^S is the volume of the hypersphere, and V_d^C is the volume of the hypercube. The crosses in the different figures are generated by each coordinates following a uniform distribution $\mathcal{U}(0, R)$ (so on average they have a value of $R/2$). When d increases, the ratio between the hypersphere and the hypercube becomes smaller and smaller, so that the majority of the volume of the hypercube lies in the corners, this means that the majority of crosses will be equally far from the center of the hypersphere (for instance a nearest neighbors algorithm would not work at all!). The intuitions we have easily in 2D are not valid anymore, so we can imagine why it is difficult to build good classifiers in high dimensions.



High-dimensional data (1/2)

Curse of dimensionality

- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.



$$V_d^S = \frac{\pi^{d/2} R^d}{\Gamma(d/2 + 1)} \text{ versus } V_d^C = (2R)^d$$

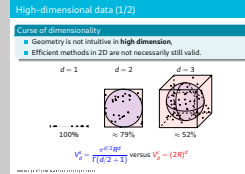
see <https://youtu.be/dZrGXYty3qc?t=533>

2026-08-24

Course 2: Supervised Learning

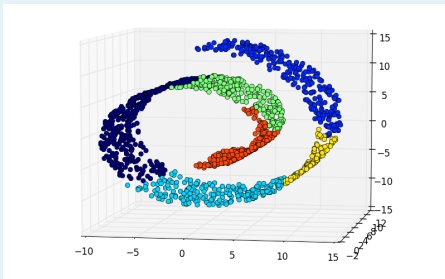
└ Challenges of supervised learning

└ High-dimensional data (1/2)



The point here is to show that when the dimension increases, the space tends to be more and more "empty". V_d^S is the volume of the hypersphere, and V_d^C is the volume of the hypercube. The crosses in the different figures are generated by each coordinates following a uniform distribution $\mathcal{U}(0, R)$ (so on average they have a value of $R/2$). When d increases, the ratio between the hypersphere and the hypercube becomes smaller and smaller, so that the majority of the volume of the hypercube lies in the corners, this means that the majority of crosses will be equally far from the center of the hypersphere (for instance a nearest neighbors algorithm would not work at all!). The intuitions we have easily in 2D are not valid anymore, so we can imagine why it is difficult to build good classifiers in high dimensions.

Riemannian manifolds



The natural space of data may not always be suited to represent data!
⇒ This is why we generally use feature descriptions, or embeddings, to represent data!

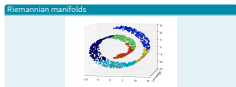
2026-08-24

Course 2: Supervised Learning

└ Challenges of supervised learning

└ High-dimensional data (2/2)

High-dimensional data (2/2)



The natural space of data may not always be suited to represent data!
⇒ This is why we generally use feature descriptions, or embeddings, to represent data!

This is a “swiss roll”. The points live in 3D but do not fill it: they lie on a sheet that has been rolled up, i.e. a **2D surface** embedded in 3D. The data is intrinsically simpler than the space describing it. Think of the flat cake before rolling; the useful coordinates are those of the flat sheet, not x, y, z .

Consequence on distances, hence on most algorithms: two points sitting on neighbouring layers of the roll are close in the Euclidean sense, but very far apart along the sheet, as an ant walking on the surface would find. A method relying on the naive distance, KNN typically, will call them similar and be wrong.

In general, data rarely fills its representation space: it concentrates on a lower-dimensional structure called a **manifold** (locally Euclidean, globally curved; like the surface of the Earth). Hence the conclusion of the slide: **features** and **embeddings** aim at re-expressing the data in a space where that structure is flattened, so that, hopefully, ordinary distances and simple classifiers become meaningful again.

Computation time

Example on ImageNet, simply going through all images:

- $n = 10,000,000$, $d \approx 1,000,000$,
- $\approx 10^{13}$ elementary operations,
- ≈ 2045 on a modern processor.

Scalability

- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often untractable,
- Solutions must be computationally reasonable, which is the true challenge today.

- $n = 10.000.000$, $d \approx 1.000.000$,
- $\approx 10^{13}$ elementary operations,
- $\approx 2h45$ on a modern processor.

The cost of the data. For ImageNet: about 10^7 images of roughly 10^6 values, so 10^{13} operations just to *read* every value once (around three hours, for one single pass doing nothing!). Training needs many passes, each far more than one operation per value. Dataset size is a hard practical constraint, which is why your project dataset should stay in the 1 000–10 000 range.

The cost of the search. Learning selects a function among all those the model can represent, and that set grows extremely fast with the number of parameters: exhaustive search is impossible. So we need an heuristic: an automatic procedure able to find a “good enough” solution (such as gradient descent, which finds a local optimum with no guarantee of being global).

Hence the message: a method is judged on the quality it reaches **within an acceptable computational budget**. Reflex for the lab: try your training on a small subsample before launching it on everything.

Computation time

Example on ImageNet, simply going through all images:

- $n = 10,000,000$, $d \approx 1,000,000$,
- $\approx 10^{13}$ elementary operations,
- $\approx 2h45$ on a modern processor.

Scalability

- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often **untractable**,
- Solutions must be computationally reasonable, which is the true challenge today.

- $n = 10.000.000$, $d \approx 1.000.000$,
- $\approx 10^{13}$ elementary operations,
- $\approx 2h45$ on a modern processor.

- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often **untractable**,
- Solutions must be computationally reasonable, which is the true challenge today.

The cost of the data. For ImageNet: about 10^7 images of roughly 10^6 values, so 10^{13} operations just to *read* every value once (around three hours, for one single pass doing nothing!). Training needs many passes, each far more than one operation per value. Dataset size is a hard practical constraint, which is why your project dataset should stay in the 1 000–10 000 range.

The cost of the search. Learning selects a function among all those the model can represent, and that set grows extremely fast with the number of parameters: exhaustive search is impossible. So we need an heuristic: an automatic procedure able to find a “good enough” solution (such as gradient descent, which finds a local optimum with no guarantee of being global).

Hence the message: a method is judged on the quality it reaches **within an acceptable computational budget**. Reflex for the lab: try your training on a small subsample before launching it on everything.

1 Learning from labeled examples

2 Challenges of supervised learning

3 Evaluation: ground truth and metrics

Evaluation: ground truth vs. estimation

Ground truth y

The **reference** answer: given by a human expert, a sensor, or a measurement protocol.
⇒ what we *assume* to be correct.

Estimation $\hat{y} = f(x)$

The answer **produced by the model**.
⇒ available for any x , even without label.

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat
	✓	✓	✗	✓	✓

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Evaluation: ground truth vs. estimation

Ground truth y	Estimation $\hat{y} = f(x)$
The reference answer: given by a human expert, a sensor, or a measurement protocol. ⇒ what we assume to be correct.	The answer produced by the model. ⇒ available for any x , even without label.
Test example	x_1 x_2 x_3 x_4 x_5
Ground truth y	cat dog cat dog cat
Estimation $\hat{y}$	cat dog dog dog cat
	✓ ✓ ✗ ✓ ✓

Two objects never to be confused.

Ground truth y is the label attached to an example: an annotator wrote “cat” under this image, a doctor wrote “benign” in this report. It exists only for the examples we have annotated, and producing it is usually the most expensive part of a project.

Estimation $\hat{y} = f(x)$ is what the model outputs. It exists for *any* input, including inputs nobody ever labeled — which is precisely what makes the model useful. The **hat** is the standard notation for an *estimate*, hence $\hat{y}$ for the prediction and y for the reference (the same convention as `sklearn's y_true / y_pred`).

Evaluation puts the two side by side and counts. It only makes sense on the **test set**: on the training set the comparison measures memorisation, not generalisation.

Finally, the ground truth is **not** the truth: annotators disagree, labels contain errors, many tasks are ambiguous. This caps the performance any model can reach, and a model at 100% is more often the sign of a bug — typically a test set leak — than of a perfect model.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

⇒ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k ?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Why do we need metrics?

A metric takes the whole list of pairs $(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ on the test set and returns a few scalars. Without it you would inspect every prediction by hand, impossible beyond a few dozen examples, with no way to say that model A beats model B. Note the third use listed on the slide: a single number is meaningless in isolation. What informs is a **comparison** — train vs. test (overfitting), model vs. model, or model vs. a trivial baseline such as “always predict the majority class”. Always compute that baseline first: if your classifier does not beat it, it has learned nothing.

Loss vs. metric is a classical confusion. The loss is an *optimisation* tool: cross-entropy is used because it is smooth and differentiable, but “a cross-entropy of 0.31” means nothing to a client. The metric is a *communication* tool: “the model detects 92% of the faulty parts” is actionable. They are computed on different data as well. Consequence: minimising the loss does not automatically maximise the metric you care about.

Why do we need metrics?

The problem
Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.
⇒ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- Choosing between two algorithms (KNN or a decision tree?),
- Tuning hyperparameters (which value of k ?),
- Detecting overfitting (compare train score vs. test score),
- Deciding whether the model is good enough to be used at all.

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$	Metric
What the algorithm minimises during training. Must be optimisable, not necessarily interpretable.	What you report and interpret, on the test set. Must be meaningful for the application.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

⇒ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k ?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Why do we need metrics?

A metric takes the whole list of pairs $(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ on the test set and returns a few scalars. Without it you would inspect every prediction by hand, impossible beyond a few dozen examples, with no way to say that model A beats model B. Note the third use listed on the slide: a single number is meaningless in isolation. What informs is a **comparison** — train vs. test (overfitting), model vs. model, or model vs. a trivial baseline such as “always predict the majority class”. Always compute that baseline first: if your classifier does not beat it, it has learned nothing.

Loss vs. metric is a classical confusion. The loss is an *optimisation* tool: cross-entropy is used because it is smooth and differentiable, but “a cross-entropy of 0.31” means nothing to a client. The metric is a *communication* tool: “the model detects 92% of the faulty parts” is actionable. They are computed on different data as well. Consequence: minimising the loss does not automatically maximise the metric you care about.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.
⇒ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- Choosing between two algorithms (KNN or a decision tree?),
- Tuning hyperparameters (which value of k ?),
- Detecting overfitting (compare train score vs. test score),
- Deciding whether the model is good enough to be used at all.

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

⇒ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k ?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Why do we need metrics?

A metric takes the whole list of pairs $(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ on the test set and returns a few scalars. Without it you would inspect every prediction by hand, impossible beyond a few dozen examples, with no way to say that model A beats model B. Note the third use listed on the slide: a single number is meaningless in isolation. What informs is a **comparison** — train vs. test (overfitting), model vs. model, or model vs. a trivial baseline such as “always predict the majority class”. Always compute that baseline first: if your classifier does not beat it, it has learned nothing.

Loss vs. metric is a classical confusion. The loss is an *optimisation* tool: cross-entropy is used because it is smooth and differentiable, but “a cross-entropy of 0.31” means nothing to a client. The metric is a *communication* tool: “the model detects 92% of the faulty parts” is actionable. They are computed on different data as well. Consequence: minimising the loss does not automatically maximise the metric you care about.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

⇒ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k ?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss vs. metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What you report and interpret, on the test set. Must be meaningful for the application.

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}_i = y_i]$$

Test example	x ₁	x ₂	x ₃	x ₄	x ₅
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat
	✓	✓	✗	✓	✓

$$\text{accuracy} = 4/5 = 80\%$$

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ A first metric: accuracy

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}_i = y_i]$$

Test example	x ₁	x ₂	x ₃	x ₄	x ₅
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat

accuracy = 4/5 = 80%

The simplest and most widely used metric: count the correct predictions and divide by the number of examples. The notation $\mathbf{1}[\cdot]$ is the **indicator function**, worth 1 when the condition inside is true and 0 otherwise, so the sum is just the number of correct answers.

It ranges from 0 to 1 (often given in %), and it is the quantity returned by `score` on any `sklearn` classifier, so you will meet it constantly in the lab.

Two reflexes before reading any accuracy. First, **on which set** was it computed? A training accuracy and a test accuracy tell completely different stories, and the gap between them is your overfitting indicator. Second, **what would a trivial model score?** With two balanced classes, random guessing already gives 50%, so 55% is nearly worthless; the relevant baseline is always “always predict the majority class”.

Accuracy also assumes that all mistakes are equally serious and that all classes matter equally — two assumptions that are frequently false, as the next slide shows.

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}_i = y_i]$$

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat
	✓	✓	✗	✓	✓

$$\text{accuracy} = 4/5 = 80\%$$

2026-08-24

- Course 2: Supervised Learning
 - Evaluation: ground truth and metrics
 - A first metric: accuracy

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}_i = y_i]$$

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat

accuracy = 4/5 = 80%

The simplest and most widely used metric: count the correct predictions and divide by the number of examples. The notation $\mathbf{1}[\cdot]$ is the **indicator function**, worth 1 when the condition inside is true and 0 otherwise, so the sum is just the number of correct answers.

It ranges from 0 to 1 (often given in %), and it is the quantity returned by `score` on any `sklearn` classifier, so you will meet it constantly in the lab.

Two reflexes before reading any accuracy. First, **on which set** was it computed? A training accuracy and a test accuracy tell completely different stories, and the gap between them is your overfitting indicator. Second, **what would a trivial model score?** With two balanced classes, random guessing already gives 50%, so 55% is nearly worthless; the relevant baseline is always “always predict the majority class”.

Accuracy also assumes that all mistakes are equally serious and that all classes matter equally — two assumptions that are frequently false, as the next slide shows.

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}_i = y_i]$$

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat
	✓	✓	✗	✓	✓

$$\text{accuracy} = 4/5 = 80\%$$

2026-08-24

- Course 2: Supervised Learning
 - Evaluation: ground truth and metrics
 - A first metric: accuracy

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}_i = y_i]$$

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat

accuracy = 4/5 = 80%

The simplest and most widely used metric: count the correct predictions and divide by the number of examples. The notation $\mathbf{1}[\cdot]$ is the **indicator function**, worth 1 when the condition inside is true and 0 otherwise, so the sum is just the number of correct answers.

It ranges from 0 to 1 (often given in %), and it is the quantity returned by `score` on any `sklearn` classifier, so you will meet it constantly in the lab.

Two reflexes before reading any accuracy. First, **on which set** was it computed? A training accuracy and a test accuracy tell completely different stories, and the gap between them is your overfitting indicator. Second, **what would a trivial model score?** With two balanced classes, random guessing already gives 50%, so 55% is nearly worthless; the relevant baseline is always “always predict the majority class”.

Accuracy also assumes that all mistakes are equally serious and that all classes matter equally — two assumptions that are frequently false, as the next slide shows.

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.



A very lazy model: "everybody is healthy"

- Accuracy = $990/1000 = 99\%$,
- Number of sick patients detected: 0.

⇒ we need metrics that look at **each class separately**,
and at the **type** of error being made.

2026-08-24

Course 2: Supervised Learning

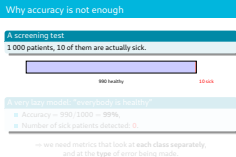
└ Evaluation: ground truth and metrics

└ Why accuracy is not enough

Reproduce this yourself with `sklearn` on an imbalanced `make_blobs` dataset.
Class imbalance. When one class dominates, accuracy is dominated by it. The "everybody is healthy" model is useless yet scores 99%. Realistic tasks are very often imbalanced: fraud, rare disease screening, defect detection, spam. Your own project data probably is too — check the class counts before interpreting any score.

Errors are not symmetric. Accuracy weights all mistakes equally, but these two have nothing in common. Telling a healthy patient he may be sick (*false positive*) costs an extra examination; telling a sick patient he is healthy (*false negative*) may cost his life. A good evaluation shows them separately.

Both observations lead to the next slides: precision and recall each look at one type of error, and the confusion matrix shows them all. In the lab you get both with `classification_report` and `confusion_matrix`.



Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.



A very lazy model: "everybody is healthy"

- Accuracy = $990/1000 = 99\%$,
- Number of sick patients detected: **0**.

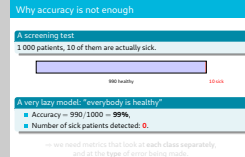
⇒ we need metrics that look at **each class separately**,
and at the **type** of error being made.

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Why accuracy is not enough



Reproduce this yourself with `sklearn` on an imbalanced `make_blobs` dataset.
Class imbalance. When one class dominates, accuracy is dominated by it. The "everybody is healthy" model is useless yet scores 99%. Realistic tasks are very often imbalanced: fraud, rare disease screening, defect detection, spam. Your own project data probably is too — check the class counts before interpreting any score.

Errors are not symmetric. Accuracy weights all mistakes equally, but these two have nothing in common. Telling a healthy patient he may be sick (*false positive*) costs an extra examination; telling a sick patient he is healthy (*false negative*) may cost his life. A good evaluation shows them separately.

Both observations lead to the next slides: precision and recall each look at one type of error, and the confusion matrix shows them all. In the lab you get both with `classification_report` and `confusion_matrix`.

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.



A very lazy model: "everybody is healthy"

- Accuracy = $990/1000 = 99\%$,
- Number of sick patients detected: **0**.

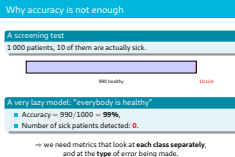
⇒ we need metrics that look at **each class separately**,
and at the **type** of error being made.

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Why accuracy is not enough



Reproduce this yourself with `sklearn` on an imbalanced `make_blobs` dataset.
Class imbalance. When one class dominates, accuracy is dominated by it. The "everybody is healthy" model is useless yet scores 99%. Realistic tasks are very often imbalanced: fraud, rare disease screening, defect detection, spam. Your own project data probably is too — check the class counts before interpreting any score.

Errors are not symmetric. Accuracy weights all mistakes equally, but these two have nothing in common. Telling a healthy patient he may be sick (*false positive*) costs an extra examination; telling a sick patient he is healthy (*false negative*) may cost his life. A good evaluation shows them separately.

Both observations lead to the next slides: precision and recall each look at one type of error, and the confusion matrix shows them all. In the lab you get both with `classification_report` and `confusion_matrix`.

Precision and recall

Definition (for **one** class, chosen as "positive")

$$\text{precision} = \frac{TP}{TP + FP}$$

Among the examples **predicted** positive, how many really are?

$$\text{recall} = \frac{TP}{TP + FN}$$

Among the **truly** positive examples, how many are found?

TP: true positives, FP: false positives (*false alarms*), FN: false negatives (*missed*).

Test example	x ₁	x ₂	x ₃	x ₄	x ₅
Ground truth y	cat	dog	cat	dog	cat
Estimation ŷ	cat	dog	dog	dog	cat

With "cat" as positive: TP = 2, FP = 0, FN = 1
precision = 2/2 = **100%** but recall = 2/3 ≈ **67%**

F1-score: summarising both in one number

$$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad \text{here } F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx \mathbf{80\%}$$

Harmonic mean $\Rightarrow$ collapses as soon as **one** of the two is low.

2026-08-24

Course 2: Supervised Learning

Evaluation: ground truth and metrics

Precision and recall

Each of the two isolates **one** type of error, which is exactly what accuracy failed to do.

Precision is penalised by **false alarms** (FP): the model shouted "cat" when there was none. Recall is penalised by **misses** (FN): a cat went unnoticed. Here the model never claims a cat wrongly (precision 100%) but lets one cat slip through (recall 67%).

Both are defined **relative to a chosen positive class**. Swap the roles and the numbers change: taking "dog" as positive gives a precision of 2/3 and a recall of 2/2 on the very same predictions. Whenever someone quotes a precision, ask "of which class?".

The two are in tension: declaring everybody positive gives a recall of 100% and a terrible precision. Hence they are reported **together**, often summarised by the **F1-score**, their harmonic mean $2PR/(P+R)$, low as soon as *one* of the two is low. Which one matters most is an application decision: a medical application favours recall, a spam filter favours precision.

Precision and recall

Definition (for **one** class, chosen as "positive")

precision = $\frac{TP}{TP + FP}$ recall = $\frac{TP}{TP + FN}$

Among the examples **predicted** positive, how many really are? Among the **truly** positive examples, how many are found?

TP: true positives, FP: false positives (false alarms), FN: false negatives (missed).

Ground truth y	cat	dog	cat	dog	cat
Estimation ŷ	cat	dog	dog	dog	cat

With "cat" as positive: TP = 2, FP = 0, FN = 1
precision = 2/2 = 100% but recall = 2/3 ≈ 67%

With "dog" as positive: TP = 2, FP = 0, FN = 1
precision = 2/3 ≈ 67% but recall = 2/2 = 100%

$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$ here $F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx 80\%$

Harmonic mean $\Rightarrow$ collapses as soon as one of the two is low.

Precision and recall

Definition (for **one** class, chosen as "positive")

$$\text{precision} = \frac{TP}{TP + FP}$$

Among the examples **predicted** positive, how many really are?

$$\text{recall} = \frac{TP}{TP + FN}$$

Among the **truly** positive examples, how many are found?

TP: true positives, FP: false positives (*false alarms*), FN: false negatives (*missed*).

Test example	x ₁	x ₂	x ₃	x ₄	x ₅
Ground truth y	cat	dog	cat	dog	cat
Estimation ŷ	cat	dog	dog	dog	cat

With "cat" as positive: TP = 2, FP = 0, FN = 1
precision = 2/2 = **100%** but recall = 2/3 ≈ **67%**

F1-score: summarising both in one number

$$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad \text{here } F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx \mathbf{80\%}$$

Harmonic mean ⇒ collapses as soon as **one** of the two is low.

2026-08-24

Course 2: Supervised Learning

Evaluation: ground truth and metrics

Precision and recall

Each of the two isolates **one** type of error, which is exactly what accuracy failed to do.

Precision is penalised by **false alarms** (FP): the model shouted "cat" when there was none. Recall is penalised by **misses** (FN): a cat went unnoticed. Here the model never claims a cat wrongly (precision 100%) but lets one cat slip through (recall 67%).

Both are defined **relative to a chosen positive class**. Swap the roles and the numbers change: taking "dog" as positive gives a precision of 2/3 and a recall of 2/2 on the very same predictions. Whenever someone quotes a precision, ask "of which class?".

The two are in tension: declaring everybody positive gives a recall of 100% and a terrible precision. Hence they are reported **together**, often summarised by the **F1-score**, their harmonic mean $2PR/(P+R)$, low as soon as *one* of the two is low. Which one matters most is an application decision: a medical application favours recall, a spam filter favours precision.

Precision and recall

Definition (for **one** class, chosen as "positive")

$$\text{precision} = \frac{TP}{TP + FP} \quad \text{recall} = \frac{TP}{TP + FN}$$

Among the examples **predicted** positive, how many really are? Among the **truly** positive examples, how many are found?

TP: true positives, FP: false positives (*false alarms*), FN: false negatives (*missed*).

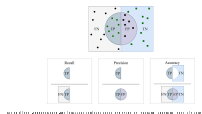
Test example	x ₁	x ₂	x ₃	x ₄	x ₅
Ground truth y	cat	dog	cat	dog	cat
Estimation ŷ	cat	dog	dog	dog	cat

With "cat" as positive: TP = 2, FP = 0, FN = 1
precision = 2/2 = **100%** but recall = 2/3 ≈ **67%**

F1-score: summarising both in one number

$$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad \text{here } F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx \mathbf{80\%}$$

Harmonic mean ⇒ collapses as soon as **one** of the two is low.

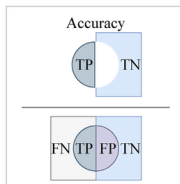
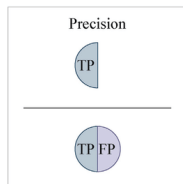
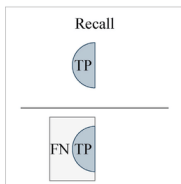
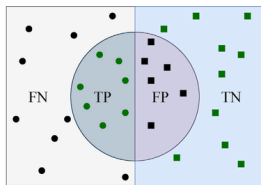


2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Precision and recall: the intuition



https://www.researchgate.net/publication/346129022-Overview_of_Machine_Learning_Part_1/figures

The picture is a one-class (positive vs. negative) problem, and it is worth reading properly: the **circle** is the set of examples the model *declared* positive, the **shaded region** is the set of examples that *are* positive, and their intersection is *TP*.

From there the two definitions become visual. **Precision** is the fraction of the *circle* falling inside the shaded region: shrink the circle around the sure cases and it goes up. **Recall** is the fraction of the *shaded region* covered by the circle: inflate the circle and it goes up. This is exactly why the two pull in opposite directions.

Accuracy, by contrast, counts everything correctly placed *including the whole outside area*, the true negatives. When negatives vastly outnumber positives that outside area dominates the count, and the circle and the shaded region become invisible details: this is precisely the 99% screening model of a few slides ago.

Useful habit: when a result looks surprising, redraw this diagram with your own numbers. It shows immediately whether the model is over-cautious (small circle) or trigger-happy (large circle).

2026-08-24

Course 2: Supervised Learning

- └ Evaluation: ground truth and metrics

└ What about several classes?

One class at a time ("one-vs-rest")

For each class c : treat c as **positive** and **all the others** as **negative**.
 $\Rightarrow$ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 = 67\%$
class "dog"	$2/3 = 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes $\rightarrow$ every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the TP, FP, FN of all classes.

(micro: precision = 80%, macro: precision = 80%)

Precision and recall are defined for one positive class against the rest, so with C classes we do it C times: the **one-vs-rest** decomposition. The result is a small table, one line per class, usually more informative than any single number since it shows which class the model handles badly.

- To obtain one number the per-class values are averaged, and the choice of averaging matters as much as the metric.

Macro gives every class the same weight whatever its size, so a tiny class handled terribly visibly drags the score down — generally what you want on imbalanced data. **Weighted** weights each class by its number of examples, so the majority class dominates again, hiding exactly what you wanted to see. **Micro** pools all the counts; in a single-label problem every error is an *FP* for one class and an *FN* for another, so micro precision, micro recall and accuracy coincide.

Reading `classification_report`: one line per class (precision, recall, F1, support), then `macro avg` and `weighted avg`. Read the per-class lines first, and check the support — a precision computed on three examples means nothing.

What about several classes?

One class at a time ("one-vs-rest")

For each class c : treat c as **positive** and **all the others** as negative.
⇒ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes → every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the TP , FP , FN of all classes.

Here: macro precision $\approx 83\%$, micro precision = 80%.

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ What about several classes?

What about several classes?

One class at a time ("one-vs-rest")
For each class c : treat c as **positive** and **all the others** as negative.
⇒ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes → every class counts the same
- **weighted**: mean weighted by the number of examples per class
- **micro**: pool the TP , FP , FN of all classes

Here: macro precision $\approx 83\%$, micro precision = 80%

Precision and recall are defined for one positive class against the rest, so with C classes we do it C times: the **one-vs-rest** decomposition. The result is a small table, one line per class, usually more informative than any single number since it shows which class the model handles badly.

To obtain one number the per-class values are averaged, and the choice of averaging matters as much as the metric.

Macro gives every class the same weight whatever its size, so a tiny class handled terribly visibly drags the score down — generally what you want on imbalanced data. **Weighted** weights each class by its number of examples, so the majority class dominates again, hiding exactly what you wanted to see. **Micro** pools all the counts; in a single-label problem every error is an FP for one class and an FN for another, so micro precision, micro recall and accuracy coincide.

Reading `classification_report`: one line per class (precision, recall, F1, support), then `macro avg` and `weighted avg`. Read the per-class lines first, and check the support — a precision computed on three examples means nothing.

What about several classes?

One class at a time ("one-vs-rest")

For each class c : treat c as **positive** and **all the others** as negative.
⇒ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes → every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the TP , FP , FN of all classes.

Here: macro precision $\approx 83\%$, micro precision = 80%.

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ What about several classes?

What about several classes?

One class at a time ("one-vs-rest")

For each class c : treat c as **positive** and **all the others** as negative.

⇒ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

■ **macro**: plain mean over classes → every class counts the same,

■ **weighted**: mean weighted by the number of examples per class,

■ **micro**: pool the TP , FP , FN of all classes.

Here: macro precision $\approx 83\%$, micro precision = 80%.

Precision and recall are defined for one positive class against the rest, so with C classes we do it C times: the **one-vs-rest** decomposition. The result is a small table, one line per class, usually more informative than any single number since it shows which class the model handles badly.

To obtain one number the per-class values are averaged, and the choice of averaging matters as much as the metric.

Macro gives every class the same weight whatever its size, so a tiny class handled terribly visibly drags the score down — generally what you want on imbalanced data. **Weighted** weights each class by its number of examples, so the majority class dominates again, hiding exactly what you wanted to see. **Micro** pools all the counts; in a single-label problem every error is an FP for one class and an FN for another, so micro precision, micro recall and accuracy coincide.

Reading `classification_report`: one line per class (precision, recall, F1, support), then `macro avg` and `weighted avg`. Read the per-class lines first, and check the support — a precision computed on three examples means nothing.

Confusion matrix

Confusion matrix

An easy way to visualise all the errors at once is the confusion matrix:

		Ground truth		
		+	-	
Predicted	+	True positive (TP)	False positive (FP)	Precision = $\frac{TP}{TP + FP}$
	-	False negative (FN)	True negative (TN)	
		Recall = $\frac{TP}{TP + FN}$		Accuracy = $\frac{TP + TN}{TP + FP + TN + FN}$

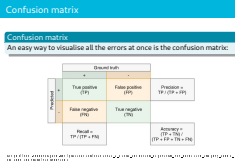
https://www.researchgate.net/publication/334840641_A_cloud_detection_algorithm_for_satellite_imagery_based_on_deep_learning/figures?lo=1

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Confusion matrix



The most complete summary of a classifier's behaviour: every metric above can be recomputed from it.

Reading it: one row per ground-truth class, one column per predicted class (or the transpose; always check the labels, conventions differ). Cell (i, j) counts the test examples of true class i assigned to class j .

Diagonal = correct. **Off-diagonal** = mistakes, and the position says which: a large value at (cat, dog) means cats taken for dogs, a different problem from dogs taken for cats.

In the two-class case the four cells are TP , FP , FN , TN , and accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$, precision = $\frac{TP}{TP+FP}$ (a column), recall = $\frac{TP}{TP+FN}$ (a row).

Unlike a single score it tells you *where* the model fails, which is what you need to improve it. In the lab, compute it on train **and** test: a clean diagonal on train with a messy one on test is the signature of overfitting.

Part 1 — Toy dataset, with sklearn

- Generate data with `make_blobs`, split it with `train_test_split`,
- The sklearn API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores $\Rightarrow$ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier. Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.

$\Rightarrow$ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for your application.

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Lab Session 2: Supervised Learning

Lab Session 2: Supervised Learning

Part 1 — Toy dataset, with sklearn

- Generate data with `make_blobs`, split it with `train_test_split`,
- The sklearn API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores $\Rightarrow$ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier. Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of your project
 $\Rightarrow$ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for your application.

The lab has the two-part structure of every lab from now on: a controlled toy dataset where everything is visible and you can check that you understood, then your own project data where nothing is clean.

Part 1 is where you build intuitions, because in 2D you can see the data and the decision boundaries; do not skip the plots. The scan over k in the KNN is the practical counterpart of the bias/variance slide: small k follows every point (high variance, perfect training score, poor test score), large k over-smooths (high bias). Plot both curves on the same figure and recognise the shape.

Part 2 brings back the questions of the beginning of this course: what exactly is your vector $\mathbf{x}$, and where does your ground truth $\mathbf{y}$ come from? Two frequent traps: strongly imbalanced classes (so accuracy is not the right metric), and features with very different scales (so KNN or SVM behave badly; look up `StandardScaler`).

Part 1 — Toy dataset, with sklearn

- Generate data with `make_blobs`, split it with `train_test_split`,
- The sklearn API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores $\Rightarrow$ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier. Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.

$\Rightarrow$ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for your application.

2026-08-24

Course 2: Supervised Learning

└ Evaluation: ground truth and metrics

└ Lab Session 2: Supervised Learning

Lab Session 2: Supervised Learning

Part 1 — Toy dataset, with sklearn

- Generate data with `make_blobs`, split it with `train_test_split`,
- The sklearn API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores $\Rightarrow$ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier. Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.
 $\Rightarrow$ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for your application.

The lab has the two-part structure of every lab from now on: a controlled toy dataset where everything is visible and you can check that you understood, then your own project data where nothing is clean.

Part 1 is where you build intuitions, because in 2D you can see the data and the decision boundaries; do not skip the plots. The scan over k in the KNN is the practical counterpart of the bias/variance slide: small k follows every point (high variance, perfect training score, poor test score), large k over-smooths (high bias). Plot both curves on the same figure and recognise the shape.

Part 2 brings back the questions of the beginning of this course: what exactly is your vector $\mathbf{x}$, and where does your ground truth $\mathbf{y}$ come from? Two frequent traps: strongly imbalanced classes (so accuracy is not the right metric), and features with very different scales (so KNN or SVM behave badly; look up `StandardScaler`).